NYC TLC Mobility Analytics Platform

End-to-End Data Engineering Solution & Architecture Documentation

---

1. Introduction

Business Context: The objective of this project is to build an end-to-end data engineering solution for a mobility analytics company. The goal is to analyze trip demand, operational patterns, and fare-related metrics for High-Volume For-Hire Vehicles (HVFHV) in New York City.

Objectives: Transform raw Parquet trip records into a reliable, scalable Data Warehouse using a Medallion Architecture. Implement a robust ETL pipeline with data quality checks, orchestration, and performance optimization. Provide actionable business insights through an interactive Apache Superset dashboard.

2. Data Sources

Dataset: NYC TLC High-Volume For-Hire Vehicle (HVFHV) Trip Records.

URL: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Selected Files: `fhvhv_tripdata_2025-*.parquet` (Jan-Dec) and `taxi_zone_lookup.csv`.

Date Range: January 2025 – December 2025.

Raw Size: Approximately 1.5 GB per month (~20 Million rows per file).

> **Limitations:** The dataset only contains completed trips. It lacks passenger/driver IDs, making it impossible to track individual users. Analysis is based on a full year of data (approx. 240M rows).

3. Architecture

Architecture Pattern: Medallion Architecture (Bronze -> Silver -> Gold) optimized into a Lakehouse Pattern.

Architecture Evolution:

The initial design followed a classic Medallion Architecture. However, to overcome local hardware constraints (PostgreSQL I/O bottlenecks when handling 20M+ raw rows), the architecture was optimized into a Lakehouse pattern to ensure scalability and performance:

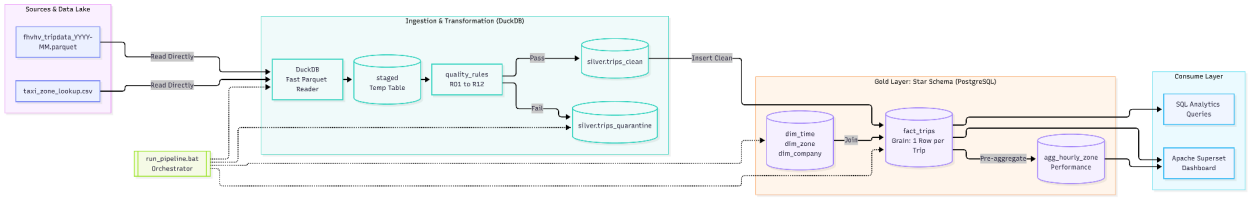
- **Bronze (Data Lake):** The raw Parquet files act as the immutable Bronze layer, eliminating the need to load millions of raw rows into a relational database.

- **Silver (Compute):** DuckDB acts as the high-performance compute engine, reading directly from the Data Lake and applying transformations in-memory (OLAP).
- **Gold (Data Warehouse):** PostgreSQL acts as the final Data Warehouse, storing only the business-ready Star Schema (OLTP).

This approach maintains the logical separation of the Medallion layers while significantly improving performance and reducing storage overhead.

Technology Choices:

- **Docker:** Containerization for isolated environments.
- **DuckDB:** High-performance compute engine for Parquet ingestion and in-memory Silver transformations.
- **PostgreSQL 16:** The core Data Warehouse storing structured Gold data.
- **Custom Batch Orchestrator (`.bat`):** Handles dependencies, retries, logging, and fail-fast mechanisms.
- **Apache Superset:** For data visualization and BI reporting



📷 *[Figure 1: Architecture Diagram / Medallion & Lakehouse Pattern]*

4. Transformations (Data Quality)

Data Quality & Cleaning:

We implemented 12 strict quality rules (R01 to R12) in the Silver Layer using DuckDB. Rows failing critical rules are redirected to a `trips_quarantine` table for traceability, while valid rows are transformed and loaded into `trips_clean`.

Schema Evolution:

The pipeline includes a normalization step that handles schema drift (e.g., older files missing `cbd_congestion_fee`). It logs missing columns (`SCHEMA_EVOLUTION` notice) and defaults them to 0 (since the fee didn't exist then).

stress Testing & Schema Drift Handling (2024 Data)

To ensure the pipeline's robustness, we conducted a stress test by ingesting a historical file (`fhvhv_tripdata_2024-06.parquet`). This file lacks the `cbd_congestion_fee` column, which was introduced by NYC TLC in January 2025.

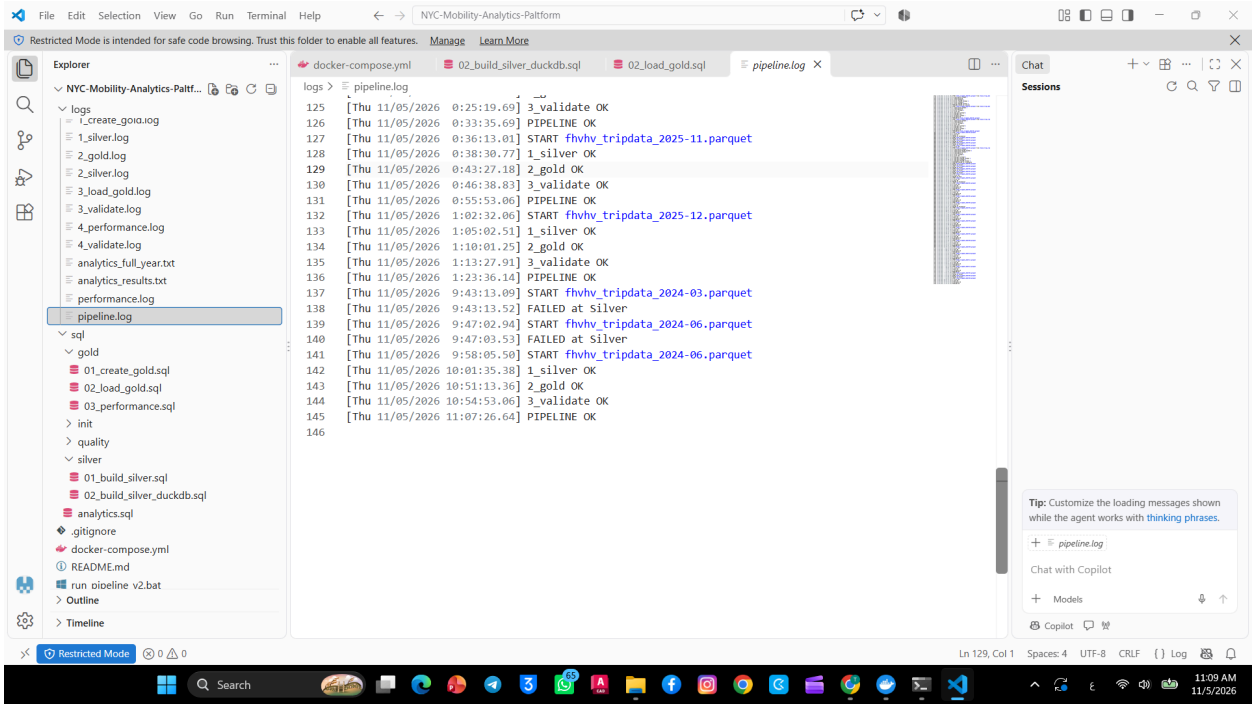
Behavior Before Fix:
The initial pipeline execution resulted in a Fail-Fast behavior. DuckDB raised a `Binder Error: Referenced column "cbd_congestion_fee" not found`, which immediately halted the pipeline. This proved that the Data Warehouse was protected from loading corrupted or incomplete schemas.

Resolution (Schema Evolution):
To support historical backfills without compromising data quality, we implemented a Schema Evolution normalization step in the Silver transformation:

1. The script dynamically detects if critical fee columns (`congestion_surcharge`, `airport_fee`, `cbd_congestion_fee`) are missing.
2. It logs a `SCHEMA_EVOLUTION` notice.
3. It automatically adds the missing columns as `NULL`, which are then handled safely in the transformation using `COALESCE(..., 0)`.

Outcome:
After the fix, the 2024 file was successfully ingested. The `cbd_congestion_fee` defaulted to `0` (indicating the fee did not exist in that period), allowing analysts to compare 2024 and 2025 data seamlessly without breaking the pipeline.

(Insert Screenshot Here: of the log showing "SCHEMA_EVOLUTION: ... is missing column cbd_congestion_fee")

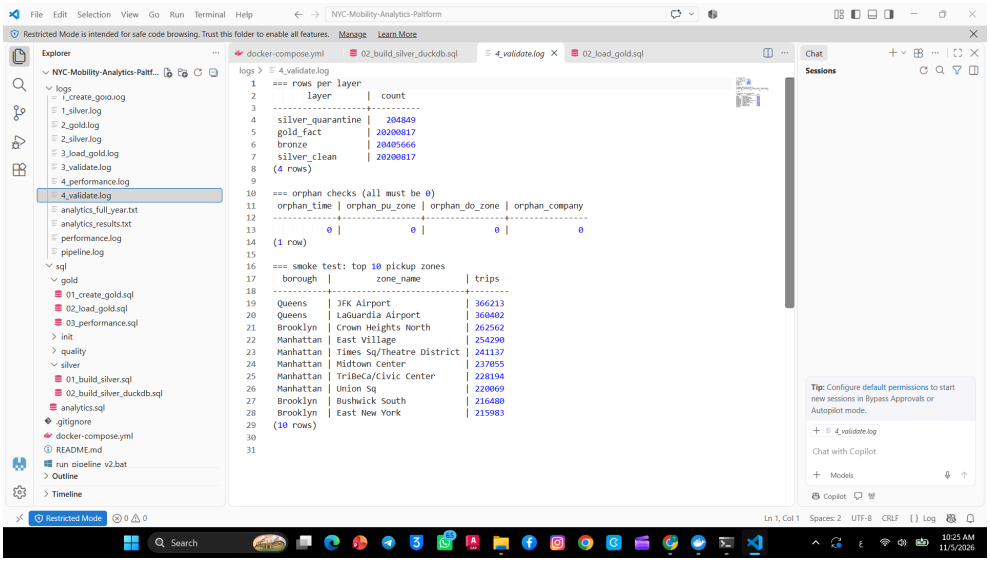


Validation:

A strict reconciliation check is performed: `Total Rows = Clean Rows + Quarantined Rows`. If the pipeline fails this check, it halts immediately (Fail-Fast).

January 2025 Execution Results :

- Total Rows (Source): 20,405,666
- Clean Rows (Valid): 20,200,817
- Quarantined Rows (Rejected): 204,849
- *Validation Check:* 20,200,817 + 204,849 = 20,405,666 (Reconciliation Passed ✅).



📷 *[Figure 2: Data Quality & Reconciliation Pipeline]*

5. Data Warehouse Model

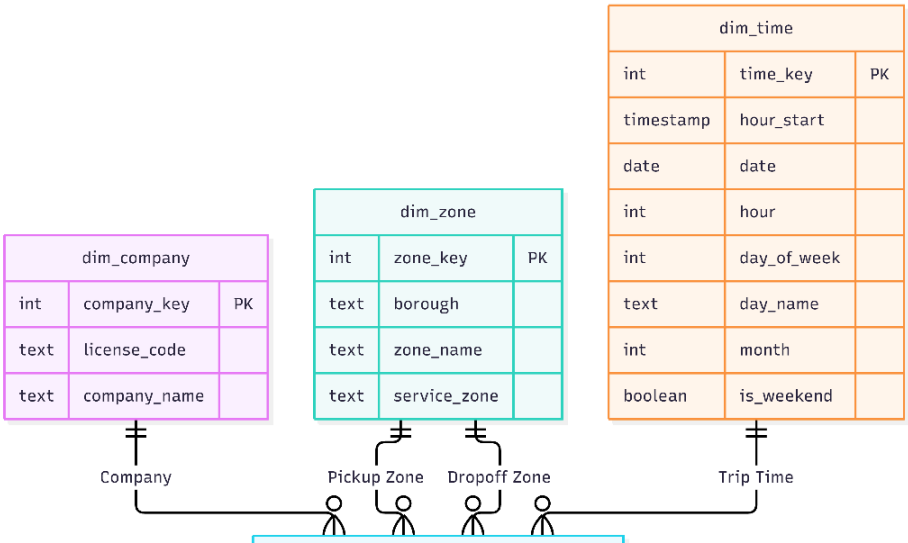
Model: Star Schema.

Fact Table: `fact_trips` (Grain: One row per completed trip, ~240M rows).

Dimensions: `dim_time` (Hourly), `dim_zone` (Role-playing: Pickup & Dropoff), `dim_company` (HVFHV Vendors).

Keys: Surrogate keys are used for dimensions. Unknown values are mapped to key `0`.

Lineage: `source_file` is kept in the fact table as a degenerate dimension for data lineage and safe reruns.



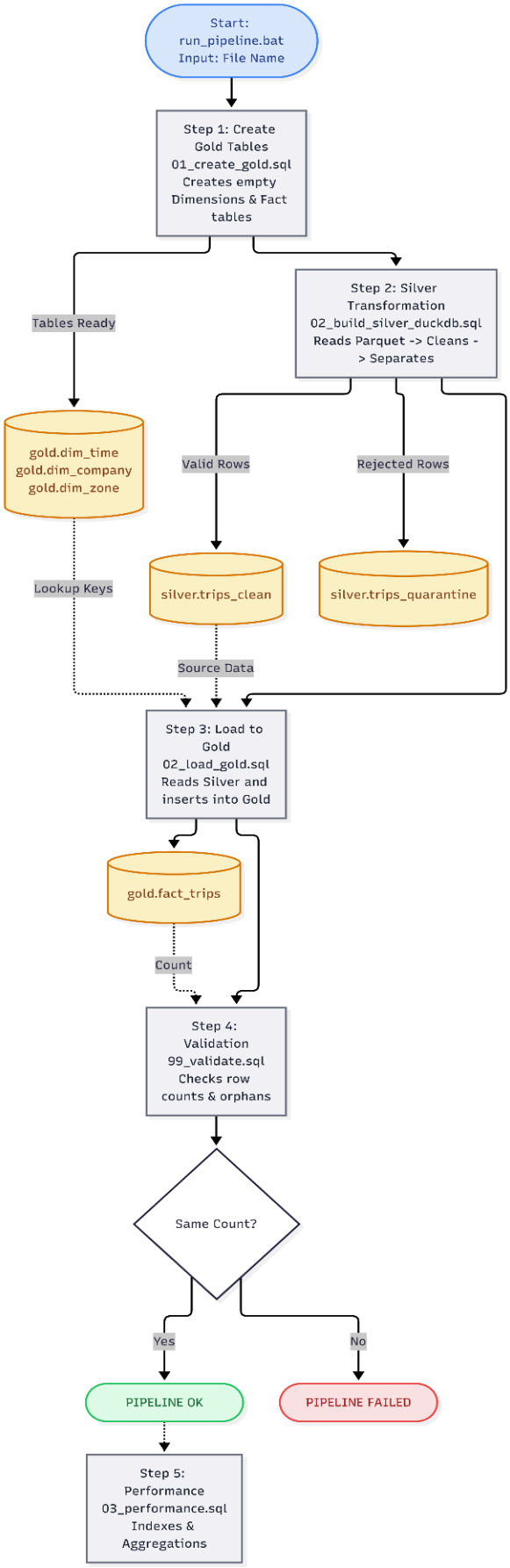
fact_trips		
bigint	trip_key	PK
int	time_key	FK
int	company_key	FK
int	pu_zone_key	FK
int	do_zone_key	FK
timestamp	request_datetime	
timestamp	pickup_datetime	
timestamp	dropoff_datetime	
numeric	trip_miles	
int	trip_seconds	
numeric	base_passenger_fare	
numeric	tolls	
numeric	bcf	
numeric	sales_tax	
numeric	congestion_surcharge	
numeric	airport_fee	
numeric	cbd_congestion_fee	
numeric	tips	
numeric	driver_pay	
int	wait_seconds	
numeric	total_paid	
boolean	shared_request	
boolean	shared_match	
boolean	access_a_ride	
boolean	wav_request	
boolean	wav_match	
text	source_file	

📷 *[Figure 3: Star Schema & Data Warehouse ERD]*

6. ETL and Orchestration

Orchestration: A Custom Batch Orchestrator (`run_pipeline_v2.bat` and `run_whole_2025.bat`) manages the workflow.

- **Dependencies:** Enforces sequential execution (DuckDB Silver -> Postgres Gold -> Validation).
- **Retries & Logging:** Implements retries and stores detailed logs in the `/logs` directory.
- **Backfills & Safe Reruns:** Parameterized by `source_file`. Uses `DELETE` before `INSERT` to ensure Idempotency.
- **Failure Recovery:** The system uses a Fail-Fast mechanism. A failed execution is captured and logged immediately.


Start:
run_pipeline.bat
Input: File Name
Step 1: Create Gold Tables
01_create_gold.sql
Creates empty Dimensions & Fact tables
Tables Ready
Step 2: Silver Transformation
02_build_silver_duckdb.sql
Reads Parquet -> Cleans -> Separates
gold.dim_time
gold.dim_company
gold.dim_zone
Valid Rows
Rejected Rows
silver.trips_clean
silver.trips_quarantine
Lookup Keys
Source Data
Step 3: Load to Gold
02_load_gold.sql
Reads Silver and inserts into Gold
gold.fact_trips
Count
Step 4: Validation
99_validate.sql
Checks row counts & orphans
Same Count?
Yes
No
PIPELINE OK
PIPELINE FAILED
Step 5: Performance
03_performance.sql
Indexes & Aggregations

📷 *[Figure 4: Pipeline Execution & Error Handling Flow]*

This was demonstrated (schema mismatch) and successfully recovered by implementing Schema Evolution handling.

7. Performance

Optimization Approach:

1. **Indexing:** B-Tree indexes on `time_key` and `pu_zone_key`.
2. **Pre-aggregation:** Summary table (`agg_hourly_zone`) for dashboard queries.

Measurements:

- **Index Impact:** Query execution time dropped from 190,373 ms to 32,002 ms (~83% faster).
- **Aggregate Impact:** Query execution time dropped from 105,409 ms to 675 ms (~99.3% faster).

8. SQL Analytics

Logs Reference:
https://github.com/tokahalawa/NYC-Mobility-Analytics-Paltform/blob/main/logs/analytics_full_year.txt:

Query 1: Trip Demand by Hour

Business Question: How does trip demand vary by hour of the day?

```
SQL
SELECT EXTRACT(HOUR FROM pickup_datetime) AS hour_of_day, COUNT(*) AS total_trips
FROM gold.fact_trips GROUP BY 1 ORDER BY 1;
```

Interpretation: Trip demand peaks between 5 PM and 7 PM (17:00 - 19:00), aligning with evening rush hour and social activities. The lowest demand is between 3 AM and 5 AM. This insight helps in dynamic fleet allocation.

Query 2: Demand by Day of Week

Business Question: Which days of the week have the highest trip volume?

```
SQL
SELECT
    CASE EXTRACT(ISODOW FROM pickup_datetime)
        WHEN 1 THEN 'Monday' WHEN 2 THEN 'Tuesday' WHEN 3 THEN 'Wednesday'
```

```
        WHEN 4 THEN 'Thursday' WHEN 5 THEN 'Friday' WHEN 6 THEN 'Saturday'
        WHEN 7 THEN 'Sunday' END AS day_name,
    COUNT(*) AS total_trips
FROM gold.fact_trips GROUP BY 1 ORDER BY 2 DESC;
```

Interpretation: Friday and Saturday show the highest demand, indicating increased social activities and nightlife during weekends. Monday has the lowest volume.

Query 3: Top 10 Pickup Zones

Business Question: Which pickup zones generate the highest trip volumes?

```
SQL
SELECT z.zone_name, z.borough, COUNT(*) AS total_trips
FROM gold.fact_trips f
JOIN gold.dim_zone z ON f.pu_zone_key = z.zone_key
GROUP BY 1, 2 ORDER BY 3 DESC LIMIT 10;
```

Interpretation: Airports (LaGuardia and JFK) dominate pickup volumes, followed by busy Manhattan and Brooklyn areas (Crown Heights, East Village, Times Square). This highlights the importance of airport logistics and dense urban zones.

Query 4: Revenue by Company

Business Question: Which HVFHV companies generate the highest revenue?

```
SQL
SELECT c.company_name, COUNT(*) AS total_trips, ROUND(SUM(f.total_paid), 2) AS
total_revenue
FROM gold.fact_trips f
JOIN gold.dim_company c ON f.company_key = c.company_key
GROUP BY 1 ORDER BY 3 DESC;
```

Interpretation: Uber dominates the market with over $5.98 Billion in revenue and 174M trips, followed by Lyft with $2.08 Billion. This shows a clear market monopoly by Uber.

Query 5: Average Trip Distance and Duration

Business Question: What is the average trip distance and duration per company?

```
SQL
SELECT c.company_name, ROUND(AVG(f.trip_miles), 2) AS avg_miles,
ROUND(AVG(f.trip_seconds) / 60.0, 2) AS avg_minutes
FROM gold.fact_trips f
```

```
JOIN gold.dim_company c ON f.company_key = c.company_key
GROUP BY 1;
```

Interpretation: Uber and Lyft have very similar operational metrics, averaging around 5 miles and 19-20 minutes per trip, indicating a standardized urban travel pattern.

Query 6: Shared vs Non-Shared Trips

Business Question: Do shared trips take longer than non-shared trips?

```
SQL
SELECT shared_match AS is_shared, COUNT(*) AS total_trips, ROUND(AVG(trip_seconds)
/ 60.0, 2) AS avg_duration_minutes
FROM gold.fact_trips GROUP BY 1;
```

Interpretation: Shared trips take significantly longer (avg ~30 mins) compared to non-shared trips (avg ~19.5 mins) due to the detours required to pick up/drop off multiple passengers.

Query 7: Tip Percentage by Borough

Business Question: Which boroughs have the highest tip percentages?

```
SQL
SELECT z.borough, ROUND(SUM(f.tips) / NULLIF(SUM(f.total_paid - f.tips), 0) * 100,
2) AS tip_percentage
FROM gold.fact_trips f
JOIN gold.dim_zone z ON f.pu_zone_key = z.zone_key
GROUP BY 1 ORDER BY 2 DESC;
```

Interpretation: Manhattan passengers tip the highest percentage (~4.11%), while Bronx passengers tip the lowest (~1.5%). This helps drivers understand which areas are more profitable in terms of tips.

Query 8: Airport Trips Trend by Hour

Business Question: How does demand for airport trips vary by hour?

```
SQL
SELECT EXTRACT(HOUR FROM pickup_datetime) AS hour_of_day, COUNT(*) AS
airport_trips
FROM gold.fact_trips f
JOIN gold.dim_zone z ON f.pu_zone_key = z.zone_key
WHERE z.zone_name IN ('JFK Airport', 'LaGuardia Airport', 'Newark Airport')
GROUP BY 1 ORDER BY 1;
```

Interpretation: Airport trips peak in the early morning and late afternoon, aligning with flight schedules and business travel patterns.

9. Dashboard

📷 *[Figure 5: Executive BI Dashboard - Apache Superset Overview]*

10. Appendix

- **GitHub Repository:** https://github.com/tokahalawa/NYC-Mobility-Analytics-Paltform
- **Setup Instructions:**
    1. `docker compose up -d`
    2. `run_whole_2025.bat`
    3. Access Superset at `localhost:8088`.